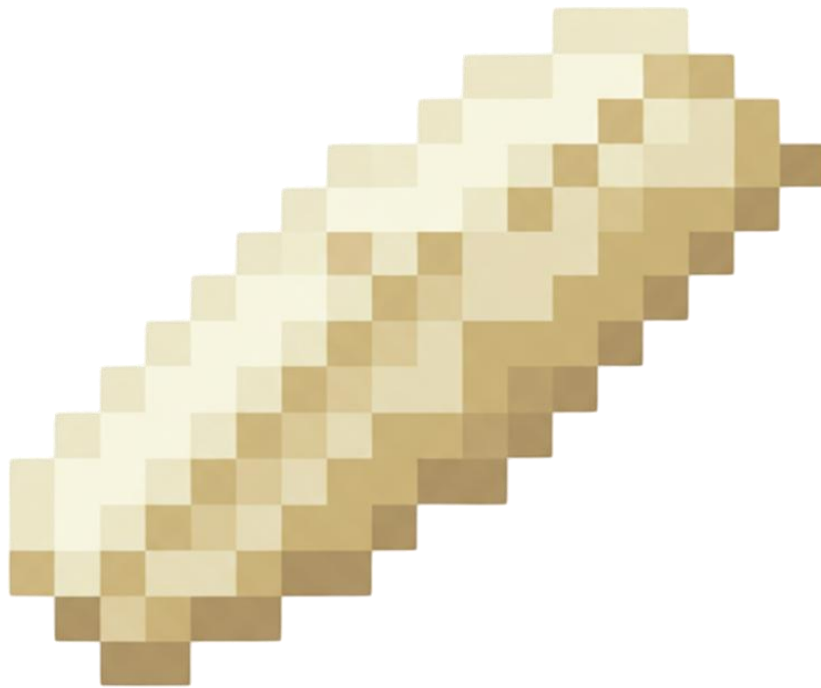


PRÁCTICA 2:

BACKEND



- Juan Manuel Valenzuela González
- Artur Vargas Carrión
- Jorge Repullo Serrano
- Eduardo González Bautista
- Rubén Oliva Zamora
- David Muñoz del Valle

- amcgil@uma.es
- arturvargas797@uma.es
- jorgers4@uma.es
- edugb@uma.es
- rubenoliva@uma.es
- davidmunvalle@uma.es

ÍNDICE DE CONTENIDO

1. REQUISITOS CONSIDERADOS	1
1.1. ENTIDADES DEL NÚCLEO Y CRUD.....	1
1.2. JUSTIFICACIÓN DE LA ENTIDAD <code>USER</code>	1
1.3. REQUISITOS DE CONSULTAS ESPECÍFICAS.....	1
1.3.1. Consultas parametrizadas	1
1.3.2. Consultas de relaciones	2
1.4. REQUISITOS ADICIONALES CUBIERTOS	2
2. TECNOLOGÍAS USADAS	4
2.1. LENGUAJE DE PROGRAMACIÓN.....	4
2.2. FRAMEWORK WEB Y API.....	4
2.3. SERVIDOR ASGI	4
2.4. BASE DE DATOS.....	4
2.4.1. MongoDB Atlas	4
2.4.2. Drivers de conexión (Motor y PyMongo)	5
2.5. VALIDACIÓN Y SERIALIZACIÓN DE DATOS	5
2.6. COMUNICACIÓN ENTRE SERVICIOS	5
2.7. CONTENEDORES Y ORQUESTACIÓN.....	5
2.7.1. Docker (contenerización).....	5
2.7.2. Docker Compose (orquestación).....	6
2.8. PAQUETE COMPARTIDO.....	6
2.9. APIS EXTERNAS (INTEGRACIÓN FUTURA)	6
2.10. HERRAMIENTAS DE DESARROLLO	7
3. BASE DE DATOS	8
3.1. CONSIDERACIONES PREVIAS AL DISEÑO DE LA BASE DE DATOS.....	8
3.1.1. Boceto preliminar de la interfaz de usuario	9
3.2. DISEÑO DE LA BASE DE DATOS	10
3.2.1. Metodología de diseño	10
3.2.2. Representación visual de la base de datos.....	10
3.2.3. Colecciones principales.....	10
3.2.3.1. Users.....	10
3.2.3.2. Calendars	11
3.2.3.3. Events	12
3.2.3.4. Notifications	14
3.2.4. Consideraciones adicionales.....	14
3.3. DESPLIEGUE EN ATLAS Y URI.....	15
4. MICROSERVICIOS	16
4.1. API GATEWAY (PUERTO 8000)	16
4.1.1. Descripción	16
4.1.2. Endpoints	16
4.1.2.1. Posibles parámetros	17
4.1.2.2. Formato de datos en petición y respuesta.....	17
4.2. USER SERVICE (PUERTO 8001)	18
4.2.1. Descripción	18
4.2.2. Endpoints	18
4.2.2.1. Posibles parámetros	18
4.2.2.2. Formato de datos en petición y respuesta.....	20

4.3.	CALENDAR SERVICE (PUERTO 8002)	21
4.3.1.	<i>Descripción</i>	21
4.3.2.	<i>Endpoints</i>	21
4.3.2.1.	Posibles parámetros	21
4.3.2.2.	Formato de datos en petición y respuesta	23
4.4.	EVENT SERVICE (PUERTO 8003)	26
4.4.1.	<i>Descripción</i>	26
4.4.2.	<i>Endpoints</i>	26
4.4.2.1.	Posibles parámetros	26
4.4.2.2.	Formato de datos en petición y respuesta	29
4.5.	NOTIFICATION SERVICE (PUERTO 8004)	32
4.5.1.	<i>Descripción</i>	32
4.5.2.	<i>Endpoints</i>	32
4.5.2.1.	Posibles parámetros	33
4.5.2.2.	Formato de datos en petición y respuesta	34
4.6.	INTEGRATION SERVICE (PUERTO 8006)	36
4.6.1.	<i>Descripción</i>	36
4.6.2.	<i>Endpoints</i>	36
4.6.2.1.	Formato de datos en petición y respuesta	37
5.	INSTALACIÓN - DOCKER	38
5.1.	SCRIPTS DE DOCKER	38
5.2.	EJECUCIÓN DEL CONTENEDOR	40
5.2.1.	<i>Ejecutar toda la aplicación (todos los servicios) - RECOMENDADO</i>	40
6.	CAMBIOS EN LA ARQUITECTURA	42
6.1.	CAMBIO DE TECNOLOGÍA CLOUD – AWS	42
6.1.1.	<i>Amazon EC2</i>	42

1. Requisitos considerados

Para el desarrollo de esta práctica, hemos implementado los **microservicios fundamentales que dan soporte al núcleo de la aplicación Basmati**, tal y como se definieron en nuestro documento de arquitectura. Los requisitos de esta práctica solicitaban la creación de servicios REST para las entidades centrales, operaciones CRUD, y un conjunto de consultas específicas.

A continuación, se detallan los requisitos que hemos cubierto:

1.1. Entidades del núcleo y CRUD

El enunciado solicitaba centrarse en **calendarios y eventos**. Hemos implementado esta funcionalidad a través de dos microservicios dedicados:

- **CalendarService (puerto 8002):** cumple con los requisitos de gestión de calendarios (RF1, RF2, RF3 de la práctica 1). Proporciona endpoints para las operaciones CRUD completas, como `POST /v1/calendars` y `DELETE /v1/calendars/{calendar_id}`.
- **EventService (puerto 8003):** cumple con los requisitos de gestión de eventos (RF5, RF6, RF7). Proporciona los endpoints CRUD, incluyendo la gestión de contenido anidado como comentarios y adjuntos (RF8, RF14), a través de endpoints como `POST /v1/events/{event_id}/comments`.

1.2. Justificación de la entidad User

El enunciado de la práctica sugería posponer la entidad Usuario. Sin embargo, tras un análisis de nuestra arquitectura, determinamos que **UserService (puerto 8001)** era un requisito importante desde esta fase inicial. Como se detalla en nuestra memoria de la práctica 1 (RF12, RF13) y en el diseño de la BD ([sección “3. Base de datos”](#)), este servicio es crítico para:

1. **Normalizar la identidad:** actúa como "fuente única de verdad" para usuarios que entran por distintos proveedores OAuth (Google, Facebook).
2. **Almacenar datos de la aplicación:** guarda información que no existe en OAuth, como las `notification_preferences`.
3. **Habilitar relaciones:** permite que el resto de microservicios resuelvan relaciones clave (ej. "calendarios creados por X") de forma eficiente.

Es de vital importancia destacar que **esta tabla no sirve para la autenticación**, pues esto se hará a través de OAuth con proveedores como Google o Facebook, tal y como se detalla en el caso de estudio.

1.3. Requisitos de consultas específicas

El enunciado exigía dos tipos de consultas avanzadas (dos por entidad), las cuales hemos implementado sistemáticamente:

1.3.1. Consultas parametrizadas

- **Para Users:**
 1. `GET /v1/users/search/by-email`: busca un usuario por su email exacto
 2. `GET /v1/users/search/by-display-name`: busca usuarios por `display_name` parcial.
 3. `GET /v1/users/search/by-oauth`: busca un usuario por sus credenciales OAuth (proveedor e id).

- **Para Calendars:**
 1. GET /v1/calendars/search/by-creator: busca por el ID del creador.
 2. GET /v1/calendars/search/by-keywords: busca por palabras clave en un array.
 3. GET /v1/calendars/search/by-visibility: busca calendarios por visibilidad.
 4. GET /v1/calendars/search/by-text: busca calendarios por texto en título, descripción y keywords.
 5. GET /v1/calendars/search/by-creator-name: busca calendarios por nombre del creador.
- **Para Events:**
 1. GET /v1/events/search/by-calendar: busca eventos por el ID del calendario.
 2. GET /v1/events/search/by-date-range: busca eventos en un rango de fechas.
 3. GET /v1/events/search/by-text: busca eventos por texto en título, descripción y localización.
 4. GET /v1/events/search/by-calendar-title: busca eventos por título de calendario.
 5. GET /v1/events/search/by-location: busca eventos por ubicación.
- **Para Notifications:**
 1. GET /v1/notifications/search/unread: busca notificaciones no leídas de un usuario.
 2. GET /v1/notifications/search/by-event: busca notificaciones asociadas a un evento.
 3. GET /v1/notifications/search/by-type: busca notificaciones por tipo.
 4. GET /v1/notifications/search/by-calendar: busca notificaciones de un calendario específico.

1.3.2. Consultas de relaciones

Hemos superado este requisito implementando múltiples consultas que explotan las relaciones (tanto referenciadas como denormalizadas) de nuestro esquema NoSQL:

- **Relación jerárquica (Calendario-Calendario):**
 1. GET /v1/calendars/{calendar_id}/children: obtiene hijos directos (relación parent_calendar_id).
 2. GET /v1/calendars/{calendar_id}/hierarchy: obtiene el árbol de descendientes completo (relación path).
- **Relación Evento-Comentario-Usuario:**
 1. GET /v1/events/{event_id}/comments/users: devuelve los usuarios que han comentado en un evento.
 2. GET /v1/events/users/{user_external_id}/commented: devuelve los eventos donde un usuario ha comentado.
- **Relación denormalizada (búsqueda):**
 1. GET /v1/search/calendars/by_creator: busca en el campo denormalizado creator_display_name.
 2. GET /v1/search/events/by_calendar_title: busca en el campo denormalizado calendar_title.

1.4. Requisitos adicionales cubiertos

Además de las entidades y consultas base, hemos implementado los servicios de soporte definidos en nuestra arquitectura para cumplir con todos los requisitos funcionales asociados al backend:

- **NotificationService (puerto 8004):** cumple con RF15 (notificación de comentarios) y RF16 (configuración de notificaciones).

- **IntegrationService (puerto 8006):** cumple con RF17 (importación de calendarios), estableciendo los *endpoints* para la importación desde Google Calendar y Teamup, aunque este servicio no esté del todo implementado porque necesita de OAuth para ello (se realizará en la siguiente práctica).

2. Tecnologías usadas

2.1. Lenguaje de programación

El lenguaje de programación principal seleccionado para el desarrollo de todos los microservicios del proyecto es **Python**.

Se ha establecido como requisito técnico indispensable el uso de la versión **3.11 o superior**. Esta decisión se fundamenta en la adopción extensiva de las características de **sintaxis moderna** que ofrece esta versión, tales como los *union types* nativos (utilizando el operador `|`) y la implementación de tipos genéricos nativos.

El uso de estas capacidades modernas de Python permite un código más limpio, legible y un sistema de tipos (typing) más robusto en toda la base del código gracias a Pydantic V2.

2.2. Framework web y API

Para la construcción de las interfaces API REST del proyecto, se ha adoptado el framework **FastAPI**.

Esta elección se fundamenta en su alto rendimiento y sus capacidades modernas. FastAPI se destaca por la **generación automática de documentación interactiva** compatible con los estándares OpenAPI, facilitando las pruebas y la integración.

El framework utiliza **Pydantic** de forma nativa para la validación de datos, lo que garantiza una definición clara y robusta de los esquemas de entrada y salida. Además, su **soporte nativo para operaciones asíncronas** (`async/await`) es crucial para la eficiencia y escalabilidad de las operaciones I/O.

Dada su idoneidad, FastAPI se utiliza de manera estandarizada como el framework base en **todos los microservicios** desarrollados y en el **API Gateway**.

2.3. Servidor ASGI

Para la ejecución de las aplicaciones FastAPI en producción, se utiliza **Uvicorn**, un servidor ASGI (Asynchronous Server Gateway Interface) de alto rendimiento.

Uvicorn es el encargado de **ejecutar todas las instancias de las aplicaciones FastAPI**, sirviendo como el punto de entrada para las solicitudes. Su arquitectura está diseñada específicamente para manejar el **soporte de operaciones asíncronas** de manera eficiente, complementando la naturaleza `async` de FastAPI.

Se ha estandarizado su uso como el **servidor de producción en todos los microservicios** del proyecto, garantizando un entorno de despliegue robusto y optimizado.

2.4. Base de datos

2.4.1. MongoDB Atlas

Como solución principal de persistencia de datos, el proyecto utiliza **MongoDB Atlas**, la plataforma de base de datos NoSQL gestionada en la nube. Esta base de datos orientada a documentos permite un almacenamiento flexible en **formato BSON** (similar a JSON).

MongoDB Atlas es la base de datos centralizada **utilizada por todos los microservicios** clave del sistema, incluyendo *UserService*, *CalendarService*, *EventService*, *NotificationService* e *IntegrationService*. La conexión a las instancias de la base de datos se gestiona de forma segura mediante una **URI de conexión** específica, la cual es proporcionada a las aplicaciones a través de variables de entorno.

2.4.2. Drivers de conexión (Motor y PyMongo)

Para interactuar con MongoDB desde las aplicaciones Python, se utiliza una pila de drivers asíncrona:

- **Motor:** es el driver principal utilizado para la comunicación con la base de datos. Se trata de un **driver asíncrono** diseñado para trabajar con `asyncio` y es fundamental para ejecutar **operaciones de base de datos de manera no bloqueante**, lo cual se alinea perfectamente con la arquitectura asíncrona de FastAPI y Uvicorn.
- **PyMongo:** es el driver oficial de MongoDB para Python y funciona como la **dependencia subyacente de Motor**. PyMongo gestiona la lógica de bajo nivel para el manejo de las conexiones y las operaciones CRUD con MongoDB.

2.5. Validación y serialización de datos

Para garantizar la integridad y estructura de los datos que fluyen a través de las APIs, se utiliza un conjunto de herramientas centrado en Pydantic:

- **Pydantic:** es la librería fundamental para la **validación y serialización de datos** en todo el proyecto. Se utiliza para definir los **schemas de request y response** de las APIs (en conjunto con FastAPI), proporcionando una **validación automática de tipos** robusta en tiempo de ejecución. El proyecto se adhiere a la sintaxis moderna de Pydantic v2, haciendo uso de `ConfigDict` para la configuración de los modelos.
- **Pydantic settings:** para la **gestión de la configuración** de las aplicaciones, se emplea esta librería. Facilita la **carga de variables de entorno** (como claves de API, URIs de conexión, etc.) en un modelo de configuración tipado, permitiendo la **validación de la configuración de los servicios** al arranque.
- **Email validator:** se utiliza esta dependencia especializada para la **validación de direcciones de correo electrónico**. Su implementación es crucial en componentes como el *UserService* para asegurar que los correos electrónicos de los usuarios son válidos y cumplen con los estándares.

2.6. Comunicación entre servicios

Para gestionar la comunicación interna dentro de la arquitectura de microservicios, se utiliza la librería **HTTPX**. Se ha seleccionado HTTPX por ser un **cliente HTTP asíncrono** moderno, lo que permite realizar solicitudes de red (operaciones I/O) de forma no bloqueante, integrándose perfectamente con el stack asíncrono de FastAPI y Uvicorn.

Su función principal se ejerce en el **API Gateway**, que utiliza HTTPX para **"proxificar" las peticiones** entrantes de los clientes hacia los microservicios internos adecuados. De esta forma, HTTPX facilita toda la **comunicación HTTP entre servicios** de manera eficiente.

2.7. Contenedores y orquestación

La arquitectura del proyecto está diseñada para ser desplegada utilizando tecnologías de contenerización, asegurando el aislamiento, la portabilidad y la consistencia entre entornos.

2.7.1. Docker (contenerización)

Se utiliza **Docker** como motor de contenedores. **Cada microservicio** del sistema (incluyendo el API Gateway) se empaqueta y ejecuta en su propio **contenedor Docker** independiente.

Cada servicio cuenta con un **Dockerfile individual** que define los pasos de construcción de su imagen. La imagen base estandarizada para todas las construcciones es **python:3.11-slim**, seleccionada por ser una imagen oficial, ligera y optimizada que contiene el intérprete de Python requerido.

2.7.2. Docker Compose (orquestración)

Para la **orquestración** de la pila de servicios (principalmente en entornos de desarrollo y pruebas), se utiliza **Docker Compose**.

El archivo de configuración `docker-compose.yml` define la aplicación completa, que consta de **seis servicios** (el API Gateway y los cinco microservicios). Este archivo gestiona aspectos cruciales de la infraestructura:

- **Definición de servicios:** levanta los 6 contenedores a partir de sus respectivas imágenes.
- **Gestión de redes:** configura y gestiona la red interna del proyecto (denominada `basmati-network`), permitiendo que los servicios se descubran y comuniquen entre sí de forma segura.
- **Gestión de configuración:** inyecta las variables de entorno necesarias en cada contenedor en el momento del arranque.
- **Dependencias:** establece la configuración de dependencias entre servicios (por ejemplo, `depends_on`) para gestionar el orden de inicio.

2.8. Paquete compartido

Para promover la reutilización de código y mantener la consistencia en toda la arquitectura de microservicios, se ha desarrollado un paquete Python interno denominado **Shared**.

Este paquete centraliza la lógica y las definiciones comunes **utilizadas por múltiples microservicios**. Durante el desarrollo, se instala mediante el modo **editable**, lo que permite que las modificaciones en el paquete compartido se reflejen instantáneamente en los servicios que lo consumen, sin necesidad de reinstalación.

El contenido principal de Shared incluye:

- **Configuración compartida (`shared/config.py`):** lógica centralizada para la carga de configuraciones.
- **Utilidades de base de datos (`shared/database.py`):** funciones comunes para la conexión y la interacción con MongoDB (utilizando Motor).
- **Schemas comunes (`shared/schemas/common.py`):** schemas de Pydantic que se comparten entre diferentes servicios (por ejemplo, estructuras de respuesta estándar).

Las **dependencias principales** de este paquete son `pydantic-settings` y `motor`, reflejando su rol en la gestión de la configuración y el acceso a datos.

2.9. APIs externas (integración futura)

El sistema está diseñado previendo la **integración futura** con servicios de calendario externos. Aunque la lógica de negocio específica para estas integraciones aún no se ha implementado, la configuración de los servicios ya contempla los requisitos de autenticación para ellas:

- **Google Calendar API:** Se ha preparado la infraestructura para una futura conexión con Google Calendar. La configuración del *IntegrationService* ya define la variable de entorno `GOOGLE_CALENDAR_API_KEY` para este fin.
- **Teamup API:** Del mismo modo, se ha planificado la integración con Teamup. La configuración ya incluye la variable de entorno `TEAMUP_API_KEY`, necesaria para la autenticación con esta API.

2.10. Herramientas de desarrollo

Para la gestión y distribución del código común del proyecto, se utiliza **Setuptools**. Esta herramienta es la responsable del **empaquetado** del módulo interno `basmati-shared`, definiéndolo como un **paquete Python instalable**. Gracias a Setuptools, el código compartido puede ser instalado (por ejemplo, mediante `pip`) como una dependencia estándar dentro de cada uno de los microservicios que lo requieren.

Como editores de código se han usado principalmente **Visual Studio Code** y **Cursor**, dos de los editores más famosos recientemente por su facilidad de uso, soporte de extensiones e integración de inteligencia artificial.

3. Base de datos

3.1. Consideraciones previas al diseño de la base de datos

Para abordar el diseño de la base de datos no relacional (NoSQL) del núcleo de Basmati, hemos adoptado un enfoque metodológico guiado por la aplicación (*application-driven design*), comenzando por la elaboración de *mockups* de la interfaz de usuario. Esta decisión es fundamental, ya que las bases de datos NoSQL están diseñadas para **optimizar los patrones de acceso a los datos** (cómo se leen y escriben) por encima de la **normalización estricta** que caracteriza a los modelos relacionales.

Al analizar los *mockups* preliminares de cada vista, hemos podido identificar con precisión qué datos necesita ver y gestionar el usuario en cada pantalla. Esto nos permite estructurar los documentos de la base de datos de forma que se alineen directamente con la información que la UI necesita mostrar.

El objetivo es diseñar un esquema que permita a la aplicación **recuperar toda la información necesaria para una vista completa en el menor número de operaciones de lectura posible** (idealmente una). Esta desnormalización intencionada (decidiendo cuándo embeber o referenciar datos) es clave para garantizar la **baja latencia y el alto rendimiento** que exige la aplicación.

En resumen, utilizar los *mockups* como punto de partida nos permite asegurar que el diseño de la base de datos y, por extensión, los microservicios REST, están alineados con los requisitos funcionales de la aplicación y la experiencia del usuario final.

A continuación, se adjuntan los bocetos iniciales preliminares que sirvieron como base para estas decisiones de diseño.

LANDING PAGE

Basmati

Organiza tu cena como si de grams de arroz se tratase

Basmati te ayuda a organizar y gestionar tu vida diaria. Desde la compra de los alimentos, pasando por la gestión de tus finanzas, hasta la organización de tus actividades. ¡Es todo lo que necesitas para vivir mejor!

Icons in the collage: calendar, shopping cart, wallet, clock, document, house, car, person, and a smartphone.

Buttons: Iniciar sesión, Registrarse

Tags: Tecnología, Finanzas, Organización, Vida Diaria

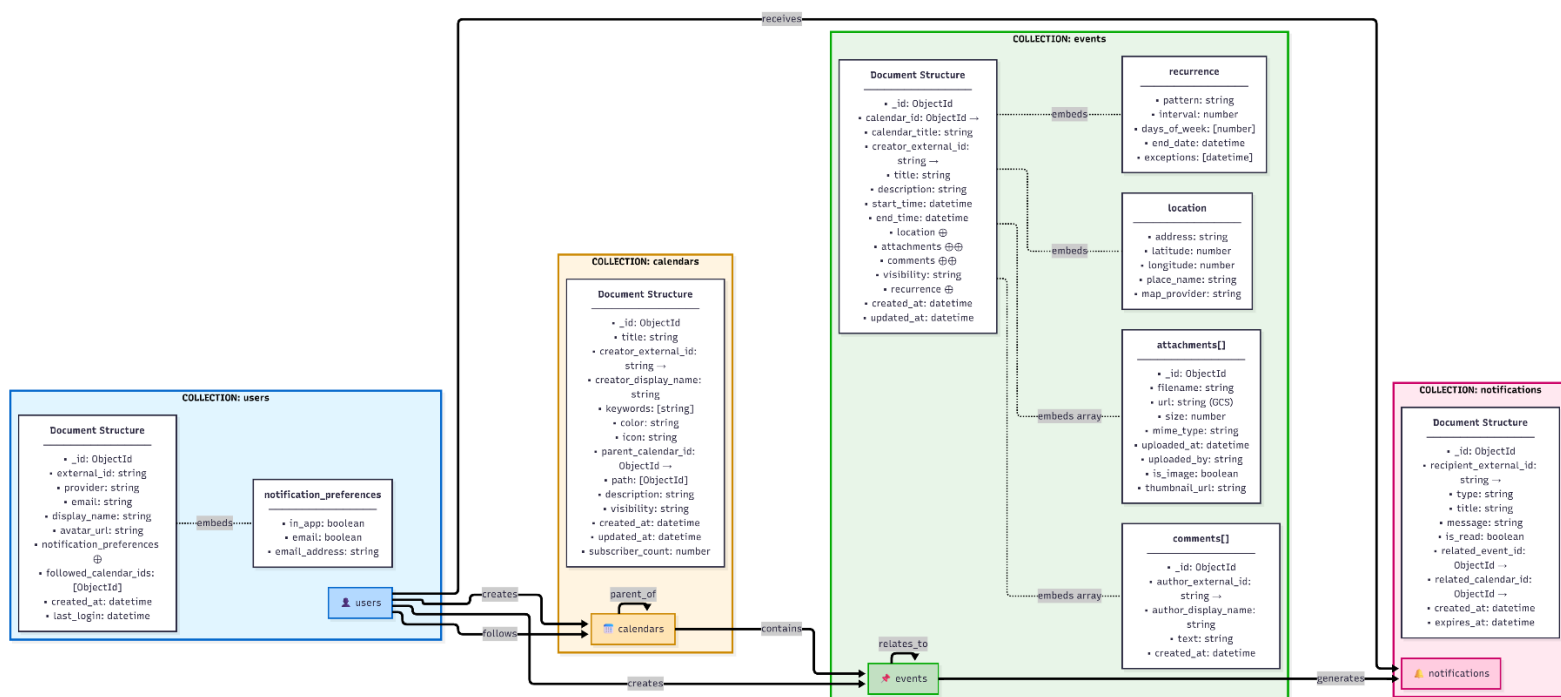


3.2. Diseño de la base de datos

3.2.1. Metodología de diseño

Adoptamos un enfoque *Application-Driven Design* comenzando por los mockups de interfaz (Fig. 1), identificando los patrones de acceso a datos que determinan las decisiones de modelado en MongoDB.

3.2.2. Representación visual de la base de datos



2 Representación visual de la base de datos NoSQL de Basmati

3.2.3. Colecciones principales

3.2.3.1. Users

A pesar de delegar la autenticación a OAuth 2.0 (Google, Facebook), mantenemos una colección `users` local por las siguientes razones críticas:

1. **Normalización de identidad multi-proveedor:** diferentes proveedores OAuth devuelven identificadores y estructuras distintas. Esta colección actúa como "fuente única de verdad" interna.
2. **Datos específicos de la aplicación:** preferencias de notificación, calendarios seguidos y configuraciones que no existen en los sistemas externos.
3. **Desacoplamiento:** si en el futuro queremos cambiar o añadir proveedores OAuth, el resto de la aplicación no se ve afectado.
4. **Eficiencia en consultas:** podemos consultar "calendarios creados por usuario X" sin realizar llamadas a APIs externas.

3.2.3.1.1. Campos principales

NOMBRE		TIPO	DESCRIPCIÓN
id		ObjectId	Identificador interno de MongoDB
external_id		string	ID único del proveedor OAuth (ej: "google-12345678")
identity_provider		string	Proveedor de identidad: "google" "facebook"
display_name		string	Nombre para mostrar en la UI
email		string	Email del usuario (para notificaciones)
profile_picture_url		string	URL de la foto de perfil (del proveedor OAuth)
created_at		datetime	Primera vez que el usuario se registró
last_login		datetime	Última sesión iniciada
notification_preferences	email_enabled	boolean	EMBEBIDO: siempre se lee junto al perfil
	in_app_enabled	boolean	
followed_calendars		[ObjectId]	EMBEBIDO: array de IDs de calendarios suscritos

3.2.3.1.2. Decisiones de diseño

- `notification_preferences` **embebido**: este subdocumento siempre se consulta junto con el perfil del usuario (para decidir cómo enviar notificaciones). Su tamaño es fijo y pequeño (~2 campos boolean), por lo que embeber es más eficiente que crear una colección separada.
- `followed_calendars` **como array de referencias**: almacenamos solo los `ObjectId` de los calendarios, no los documentos completos. Esto permite:
 - Actualizar un calendario sin tocar todos los usuarios que lo siguen
 - Mantener el documento `user` por debajo del límite de 16MB (un usuario podría seguir cientos de calendarios)
 - Consultar eficientemente "¿este usuario sigue este calendario?" con `$in`

3.2.3.1.3. Índices creados

Se crean tres índices en la colección de usuarios porque el flujo de autenticación OAuth **realiza búsquedas frecuentes en estas columnas**. Cada vez que un usuario inicia sesión con Google o Facebook, la aplicación ejecuta consultas como `db.users.findOne({ provider: "google", external_id: "...", })`. Sin índices, MongoDB **recorrería todos los documentos** ($O(n)$), causando latencia inaceptable. Con estos índices, las búsquedas son logarítmicas ($O(\log n)$) y prácticamente instantáneas.

El índice único en `external_id` garantiza que **no existan duplicados de OAuth ID**. El índice compuesto en `provider + external_id` es obligatorio porque un usuario **podría tener el mismo `external_id` en múltiples proveedores**, por lo que ambos campos son necesarios para identificar unívocamente al usuario. El índice en `email` se utiliza para **validaciones de duplicados**.

3.2.3.2. Calendars

Los calendarios son la entidad organizadora principal de Basmati. Representan colecciones temáticas de eventos y pueden formar jerarquías (ej: "Máster IA" → "Ingeniería Web" → "Prácticas").

3.2.3.2.1. Campos principales

NOMBRE		TIPO	DESCRIPCIÓN
id		ObjectId	Identificador interno de MongoDB
title		string	Título del calendario (ej: "Ingeniería Web 2025/26")
description		string	Descripción larga del calendario
creator_external_id		string	Referencia al campo <code>external_id</code> de <code>users</code>
organizer		string	Nombre del organizador (puede ser != creator)

keywords	[string]	Array de strings (ej: ["web", "málaga", "uma"]) para búsqueda
visibility	string	"public" "private" "unlisted"
parent_calendar_id	ObjectId null	ID del calendario padre o null si es calendario raíz
path	[ObjectId]	Array de ObjectId ([id_abuelo, id_padre])
created_at	datetime	Fecha inicial de creación del calendario
updated_at	datetime	Fecha de la última actualización del calendario
subscriber_count	number	Número de suscriptores del calendario

3.2.3.2.2. Decisiones de diseño

Este es el diseño más crítico de la base de datos. Tenemos **dos campos** para representar la jerarquía:

1. `parent_calendar_id`: referencia directa al padre inmediato.
 - Permite navegación "hacia arriba" eficientemente.
 - Ejemplo: calendario "Prácticas" tiene `parent_calendar_id: ObjectId("ingenieria-web")`.
2. `path`: array con todos los ancestros (del más antiguo al más reciente)
 - Permite navegación "hacia abajo" (consultas descendentes) de forma eficiente.
 - Ejemplo: calendario "Práctica 2" tiene `path: [ObjectId("master"), ObjectId("ing-web"), ObjectId("practicas")]`

El campo `path` es necesario porque sin él, para obtener "todos los subcalendarios de Máster IA recursivamente" necesitaríamos de `$graphLookup` recursivo, que es bastante lento y realiza entre 5 y 10 veces más operaciones. Con `path` es una sola consulta con un índice: `db.calendars.find({path: masterIA_id})`, que nos devolvería todos los calendarios que en su `path` tengan este id.

Por otro lado, cuando un calendario cambia de padre, debemos actualizar el `path` de todos sus descendientes. En Basmati, este caso no es común (la jerarquía de calendarios es relativamente estática), mientras que las consultas descendentes son frecuentes.

3.2.3.2.3. Índices necesarios

Para los calendarios serán necesarios **cuatro índices**. En primer lugar, uno para `creator_external_id`, con el objetivo de mostrar todos los **calendarios creados por este usuario**. El segundo será para `keywords`, para buscar calendarios por **palabras clave**, como por ejemplo por 'Málaga'. El tercer índice creado será para el `path`, con un **patrón crítico** que es el de **todos los subcalendarios** del calendario X. Por último, crearemos un índice formado por `visibility` y `created_at`, para, por ejemplo, calendarios públicos ordenados por fecha.

3.2.3.3. Events

Los eventos son el **contenido principal** de la aplicación. Representan una **actividad en un momento específico** dentro de un calendario.

3.2.3.3.1. Campos principales

NOMBRE	TIPO	DESCRIPCIÓN
id	ObjectId	Identificador interno de MongoDB
calendar_id	ObjectId	Referencia al calendario (no embebido)
title	string	Título del evento (ej: "Entrega Práctica 2")
description	string	Descripción larga del evento
start time	datetime	Hora de comienzo del evento
end time	datetime	Hora de finalización del evento
duration	number	Duración en minutos para consultas rápidas

location	address	{string, number, number}	EMBEBIDO: objeto con estructura fija que representa la localización del evento
	latitude		
	longitude		
creator_external_id		string	Referencia al campo external_id de users
comments	id	[{ObjectId, string, string, text, datetime}]	EMBEBIDO: array de comentarios de un evento.
	author_external_id		
	author_display_name		
	text		
	created_at		
attachments	id	[{ObjectId, string, string, number}]	EMBEBIDO: array de archivos adjuntos de un evento.
	filename		
	url		
	size		
created_at		datetime	Fecha inicial de creación del calendario
updated_at		datetime	Fecha de la última actualización del calendario

3.2.3.3.2. Decisiones de diseño

Es importante señalar por qué `calendar_id` es una referencia en lugar de estar embebido. Se puede dar el caso de que un calendario tenga cientos o miles de eventos, lo que supone varios problemas:

- **Límite de 16MB por documento:** un calendario con 1000 eventos, cada uno de unos 5KB con comentarios, ya superaría el límite.
- **Ineficiencia en lecturas:** para mostrar un solo evento, MongoDB debe cargar todo el documento del calendario.
- **Concurrencia:** si dos usuarios comentan eventos diferentes simultáneamente, ambos intentan modificar el mismo documento lo que implica que puede haber conflictos.

Por otro lado, tenemos que `comments` sí está embebido. Esto se debe a que los comentarios tienen características opuestas a los eventos:

- **Tamaño predecible limitado:** raramente un evento tendrá más de 100 comentarios.
- **Patrón de lecturas:** siempre se leen junto al evento.
- **Sin consultas independientes:** nunca preguntamos “todos los comentarios de la plataforma”, siempre “comentarios de este evento específico”.

Luego, debemos tener en cuenta que `author_display_name` está desnormalizado en comentarios. El motivo es claro y lo detallamos a continuación. **Sin desnormalización**, para mostrar 10 comentarios necesitaríamos:

1. Leer el evento. (1 consulta)
2. Extraer 10 `author_external_id` únicos.
3. Consultar `users` 10 veces para obtener nombres. (10 consultas)

En total, **11 consultas**.

Con desnormalización:

1. Leer el evento con comentarios embebidos. (1 consulta)

Total: **1 consulta**.

Trade-off: Si un usuario cambia su nombre, debemos actualizar todos sus comentarios. En Basmati, cambiar el nombre es **raro**, mientras que leer comentarios es **constante** (cada visualización de evento).

Otro aspecto que señalar es que `attachments` embebe **solo metadatos**, pues los archivos reales nunca se guardan en MongoDB por:

- **Tamaño:** un PDF puede pesar 50MB (y el límite de documento es 16MB).
- **Eficiencia:** MongoDB no está optimizado para servir archivos binarios.

Almacenamos solo los **metadatos** (nombre, URL, tamaño) que sí necesitamos consultar frecuentemente.

3.2.3.3.3. Índices necesarios

Para los eventos necesitaremos crear **tres índices**. El primero lo conforma `calendar_id` y `start_time`, y es compuesto porque siempre filtramos por calendario y ordenamos por fecha. El patrón sería el siguiente: eventos del calendario X en junio 2025 ordenados por fecha. El segundo índice, también compuesto, incorpora `creator_external_id` y `created_at`, para un patrón como este: eventos creados por usuario X, y los más recientes primero. Por último, un índice con `start_time`, para mostrar eventos próximos.

3.2.3.4. Notifications

Las notificaciones son **eventos temporales** (ej: "Juan comentó tu evento") que tienen un ciclo de vida completamente diferente a los eventos reales. Por eso requieren su propia colección.

3.2.3.4.1. Campos principales

NOMBRE	TIPO	DESCRIPCIÓN
<code>_id</code>	ObjectId	Identificador interno de MongoDB
<code>recipient_external_id</code>	string	Id del usuario al que va dirigida la notificación
<code>type</code>	string	Tipo de notificación ("comment" "event_update" "calendar_invite")
<code>creator_external_id</code>	string	Referencia al campo <code>external_id</code> de <code>users</code>
<code>title</code>	string	Título de la notificación
<code>message</code>	string	Contenido del mensaje de la notificación
<code>related_event_id</code>	ObjectId	Referencia al evento que generó la notificación
<code>related_calendar_id</code>	ObjectId	Referencia al calendario (para contexto)
<code>is_read</code>	boolean	Booleano que indica si se ha leído o no, para badge "3 notificaciones sin leer"
<code>created_at</code>	datetime	Fecha de creación de la notificación
<code>expires_at</code>	datetime	Para expiración automática

3.2.3.4.2. Decisiones de diseño

La decisión de no embeber las notificaciones en `users` se debe a los problemas que podría causar no haberlo diseñado así:

- Un usuario activo puede tener **miles de notificaciones**, lo que puede superar el límite de 16MB por documento.
- Para obtener solo notificaciones, **cargaríamos todo el perfil** de usuario.
- Patrón de consulta completamente diferente: "notificaciones no leídas del usuario X" no tiene nada que ver con "preferencias de usuario X".

Por otro lado, tenemos un índice TTL (Time To Live) para la expiración automática. Este índice es especial, hace que MongoDB actúe como un "recolector de basura automático". Cada minuto, MongoDB revisa los documentos y elimina los que han expirado.

3.2.3.4.3. Índices necesarios

En este caso, necesitaremos crear **dos índices**. Uno es el TTL ya mencionado antes para la expiración, y el segundo es un índice que nos ayuda a resolver un patrón muy crítico. Está compuesto por 3 campos, `recipient_external_id`, `is_read`, `created_at`, y nos ayuda con el siguiente patrón: Notificaciones no leídas de usuario X, más recientes primero. Este índice permite filtrar por usuario, por notificaciones no leídas y ordenar por fecha descendente, todo en una sola operación de índice.

3.2.4. Consideraciones adicionales

A la vista está que hemos creado varios índices compuestos. El motivo es simple, siempre que necesitamos filtrar por múltiples campos juntos creamos un índice compuesto. El siguiente ejemplo nos permite entender esto mejor:

- **Query real** de la aplicación: `db.events.find({calendar_id: "ing-web", start_time: {$gte: "2025-06-01"}}).sort({start_time: 1})`
- **Índice óptimo**: `db.events.createIndex({ "calendar_id": 1, "start_time": 1 })`

Orden de campos en índices compuestos (Regla ESR):

1. **Equality** (igualdad): `calendar_id: "valor-exacto"`
2. **Sort** (ordenación): `start_time: 1`
3. **Range** (rangos): `start_time: { $gte: ... }`

Aun así, los índices también tienen sus desventajas. A pesar de que las consultas son mucho más rápidas, ocupan espacio en disco, ralentizan escrituras, y demasiados índices pueden confundir al query planner.

3.3. Despliegue en Atlas y URI

Nuestra base de datos está desplegada utilizando **MongoDB Atlas**. Esto significa que no está alojada en un servidor local (como localhost), sino **en la nube**.

MongoDB Atlas es la plataforma oficial de **"base de datos como servicio" (DBaaS)** de MongoDB. En la práctica, esto significa:

- **Alojada en la web**: la base de datos se ejecuta en servidores gestionados por MongoDB, que a su vez utilizan proveedores de nube líderes (como AWS, Google Cloud o Azure).
- **Alta disponibilidad**: Atlas gestiona la replicación (copias de la base de datos) y las copias de seguridad automáticamente, asegurando que los datos estén seguros y siempre disponibles.
- **Accesibilidad**: se puede acceder a la base de datos de forma segura desde cualquier lugar del mundo a través de internet, siempre que se tengan las credenciales correctas y la IP esté en la lista de acceso permitida.

La URI del despliegue de la base de datos es la siguiente:

```
mongodb+srv://<db_username>:<db_password>@basmaticcluster.revmyok.mongodb.net/basmati
```

Esta no contiene la contraseña ni el usuario por cuestiones de seguridad. La URI completa se encuentra en el **.env de la aplicación**.

4. Microservicios

4.1. API Gateway (puerto 8000)

4.1.1. Descripción

Microservicio que actúa como **punto de entrada único para todas las solicitudes del cliente en Basmati**. Centraliza el enrutamiento de peticiones hacia los microservicios backend, abstrayendo la complejidad de la arquitectura distribuida y proporcionando una interfaz unificada para los consumidores de la API. Su arquitectura implementa el patrón API Gateway mediante proxy inverso HTTP asíncrono con `httpx`, permitiendo alta concurrencia y baja latencia.

Diseñado para funcionar como orquestador sin estado, el gateway no mantiene datos propios ni conexión a MongoDB; delega toda la lógica de negocio a los servicios especializados y se concentra exclusivamente en:

Responsabilidades principales:

- **Enrutamiento inteligente:** redirige solicitudes HTTP a servicios backend según prefijos de ruta (`/v1/users/*` → `UserService:8001`, `/v1/calendars/*` → `CalendarService:8002`, etc.), manteniendo compatibilidad con versionado de API.
- **Agregación de documentación OpenAPI:** consolida las especificaciones OpenAPI de todos los microservicios en un único endpoint `docs`, permitiendo a desarrolladores y consumidores explorar toda la API desde una interfaz Swagger UI unificada sin conocer la topología interna.
- **Validación de rutas y métodos:** verifica que las peticiones cumplan formatos básicos antes de reenviarlas, retornando 404 para rutas inexistentes y 405 para métodos HTTP no soportados, reduciendo carga innecesaria en servicios backend.
- **Propagación de headers y contexto:** reenvía headers de autenticación, correlación (`X-Request-ID`) y metadatos del cliente hacia los servicios downstream, manteniendo trazabilidad distribuida y preparando infraestructura para futura implementación de autenticación OAuth.
- **Manejo centralizado de errores:** captura excepciones de servicios backend (timeouts, conexiones rechazadas, errores 5xx) y las transforma en respuestas HTTP estandarizadas con códigos apropiados (503 `Service Unavailable`, 504 `Gateway Timeout`), ocultando fallos internos al cliente.
- **Timeout y circuit breaking:** configura timeouts HTTP por servicio (30s) para evitar bloqueos indefinidos.

La arquitectura del gateway se basa en configuración declarativa de servicios mediante diccionario `SERVICES` en `config.py`, mapeando prefijos de ruta a URLs de backend. El agregador OpenAPI (`core/openapi_aggregator.py`) **descarga especificaciones JSON de cada servicio al inicio, fusiona schemas/paths y genera documentación unificada bajo `docs` y `/openapi.json`.**

4.1.2. Endpoints

Endpoint	Métodos	Servicio destino	Puerto
GET /health	GET	API Gateway (propio)	8000
/v1/users/{path}	GET, POST, PUT, DELETE	UserService	8001
/v1/calendars/{path}	GET, POST, PUT, DELETE	CalendarService	8002
/v1/events/{path}	GET, POST, PUT, DELETE	EventService	8003
/v1/notifications/{path}	GET, POST, PUT, DELETE	NotificationService	8004
/v1/integrations/{path}	GET, POST, PUT, DELETE	IntegrationService	8006

4.1.2.1. Posibles parámetros

Los parámetros específicos de cada endpoint (query parameters requeridos, schemas de request body, validaciones, códigos de respuesta) están documentados en la sección correspondiente de cada microservicio:

- **UserService**: parámetros de endpoints /v1/users/*
- **CalendarService**: parámetros de endpoints /v1/calendars/*
- **EventService**: parámetros de endpoints /v1/events/*
- **NotificationService**: parámetros de endpoints /v1/notifications/*
- **IntegrationService**: parámetros de endpoints /v1/integrations/*

El API Gateway **no realiza validación de parámetros ni transformación de datos**. Todas las validaciones, schemas de request/response y lógica de negocio están implementadas en cada microservicio backend. El gateway solo se encarga del enrutamiento y la propagación transparente de la solicitud.

4.1.2.2. Formato de datos en petición y respuesta

4.1.2.2.1. Peticiones

El API Gateway reenvía todas las peticiones tal cual a los servicios backend. Los formatos de request body, query parameters y headers están definidos en cada microservicio.

4.1.2.2.2. Respuestas

El API Gateway actúa como proxy transparente: reenvía las respuestas de los servicios backend sin modificación. Los formatos de respuesta, schemas JSON y códigos HTTP están definidos en cada microservicio, por lo que tanto las respuestas exitosas como de error son propagadas sin definición. Solo hay un caso específico en el que el Gateway **genera su propia respuesta**, que es en el caso de un **error del Gateway**:

Código	Descripción
404	Servicio no encontrado en el gateway
503	No se pudo conectar con el servicio
504	Timeout al conectar con el servicio
500	Error interno del gateway

4.2. User Service (puerto 8001)

4.2.1. Descripción

Microservicio para la **gestión de usuarios** y la gestión de **autenticación OAuth** en Basmati. Gestiona el ciclo de vida de los usuarios, perfiles, preferencias de notificación y relaciones con calendarios. Su arquitectura separa responsabilidades mediante el patrón Repository, con lógica de negocio en el Service y acceso a datos aislado. Esto facilita el mantenimiento, las pruebas y la escalabilidad. El servicio utiliza **estructuras embebidas** para optimizar consultas frecuentes:

- `notification_preferences`: objeto embebido que almacena preferencias de notificación (in-app, email, email alternativo) directamente en el documento del usuario, evitando colecciones separadas.
- `followed_calendar_ids`: array de referencias a calendarios seguidos.

Entre sus responsabilidades principales se encuentran:

- **Operaciones CRUD**: creación, lectura, actualización y eliminación de usuarios con validación de esquema mediante Pydantic v2 y timestamps automáticos (`created_at`, `last_login`).
- **Búsquedas parametrizadas**: localización de usuarios por email (búsqueda exacta), nombre de visualización (regex case-insensitive parcial) y credenciales OAuth (`external_id` + `provider`), limitando resultados a 100 elementos para mantener rendimiento.
- **Gestión de preferencias**: actualización flexible de preferencias de notificación, permitiendo configuración granular por usuario (notificaciones in-app, email, dirección de email alternativa).
- **Seguimiento de calendarios**: gestión eficiente de calendarios seguidos mediante operaciones atómicas de MongoDB (`$addToSet` para agregar, `$pull` para remover), evitando condiciones de carrera y garantizando consistencia.
- **Tracking de actividad**: actualización automática de `last_login` para análisis de actividad y sesiones, útil para métricas y seguridad.

4.2.2. Endpoints

4.2.2.1. Posibles parámetros

POST `/v1/users`

Crea un nuevo usuario.

Parámetro	Tipo	Requerido	Descripción
body JSON	-	-	JSON con, al menos, los parámetros obligatorios de un usuario.

Ejemplo de uso: POST `/v1/users`

GET `/v1/users/{user_id}`

Obtener usuario por ID (ObjectId).

Parámetro	Tipo	Requerido	Descripción
<code>user_id</code>	string	Sí	ID del usuario (ObjectId como string).

Ejemplo de uso: GET `/v1/users/507f1f77bcf86cd799439011`

PUT /v1/users/{user_id}

Actualizar usuario.

Parámetro	Tipo	Requerido	Descripción
user_id	string	Sí	ID del usuario (ObjectId como string).
body JSON	-	-	JSON con los parámetros a actualizar.

Ejemplo de uso: PUT /v1/users/507f1f77bcf86cd799439011

DELETE /v1/users/{user_id}

Eliminar usuario.

Parámetro	Tipo	Requerido	Descripción
user_id	string	Sí	ID del usuario (ObjectId como string).

Ejemplo de uso: DELETE /v1/users/507f1f77bcf86cd799439011

GET /v1/users/search/by-email

Búsqueda de usuario por email exacto (parametrized query 1).

Parámetro	Tipo	Requerido	Descripción
email	string	Sí	Email exacto del usuario.

Ejemplo de uso: GET /v1/users/search/by-email?email=juan@example.com

GET /v1/users/search/by-display-name

Búsqueda de usuarios por nombre parcial case-insensitive (parametrized query 2).

Parámetro	Tipo	Requerido	Descripción
display_name	string	Sí	Parte del nombre visible del usuario.

Ejemplo de uso: GET /v1/users/search/by-display-name?display_name=Juan

GET /v1/users/search/by-oauth

Búsqueda de usuario por credenciales OAuth (external_id + provider).

Parámetro	Tipo	Requerido	Descripción
external_id	string	Sí	ID del proveedor OAuth.
provider	String	Sí	Requerido. Proveedor OAuth.

Ejemplo de uso:

GET /v1/users/search/by-oauth?external_id=google_123456789&provider=google

4.2.2.2. Formato de datos en petición y respuesta

4.2.2.2.1. Peticiones

Todos los endpoints POST y PUT utilizan el método correspondiente y pueden recibir datos mediante body JSON. Los endpoints GET y DELETE no requieren cuerpo (body) en ninguna petición.

4.2.2.2.2. Respuestas

Las respuestas del User Service siguen un formato JSON consistente. A continuación, se detallan los schemas de respuesta para cada tipo de endpoint:

UserResponse (Response Body)

```
{
  "id": "string",
  "external_id": "string",
  "provider": "string",
  "email": "string",
  "display_name": "string",
  "avatar_url": "string | null",
  "notification_preferences": {
    "in_app": "boolean",
    "email": "boolean",
    "email_address": "string | null"
  },
  "followed_calendar_ids": ["string"],
  "created_at": "datetime (ISO 8601)",
  "last_login": "datetime (ISO 8601) | null"
}
```

ResponseMessage (Response Body - solo para DELETE y follow/unfollow)

```
{
  "message": "string"
}
```

Posibles códigos de respuesta

Código	Descripción
200 OK	Operación ejecutada correctamente (GET, PUT).
201 Created	Usuario creado exitosamente (POST users).
400 Bad Request	Usuario duplicado (external_id + provider ya existe).
404 Not Found	Usuario no encontrado.
422 Unprocessable Entity	Parámetros de consulta o body inválidos (validación de Pydantic).
500 Internal Server Error	Error en el servidor o en la conexión con MongoDB.

4.3. Calendar Service (puerto 8002)

4.3.1. Descripción

Microservicio para la **gestión de calendarios** en Basmati. Es responsable de las operaciones CRUD, la organización jerárquica y las búsquedas parametrizadas. Su arquitectura separa responsabilidades mediante el patrón Repository, con lógica de negocio en el Service y acceso a datos aislado. Esto facilita el mantenimiento y las pruebas. Diseñado para soportar jerarquías multi-nivel con un campo path (array de ancestros) que permite obtener toda una jerarquía con una sola consulta, evitando múltiples queries recursivas y mejorando el rendimiento. El servicio utiliza **denormalización estratégica** para optimizar lecturas frecuentes:

- `creator_display_name`: almacena el nombre del creador directamente en el calendario, evitando joins con la colección de usuarios.
- `subscriber_count`: mantiene un contador denormalizado de suscriptores para visualizaciones rápidas sin agregaciones costosas.

Entre sus responsabilidades principales se encuentran:

- **Operaciones CRUD**: creación, lectura, actualización y eliminación de calendarios con validación de esquema mediante Pydantic v2.
- **Gestión de jerarquías**: construcción automática del campo path al crear calendarios hijos, manteniendo la integridad referencial y permitiendo consultas eficientes de descendientes.
- **Búsquedas parametrizadas**: localización de calendarios por creador (`creator_external_id`), palabras clave (regex case-insensitive en el array keywords), visibilidad (public/private/unlisted), en múltiples campos y por nombre de creador, limitando resultados a 100 elementos para mantener rendimiento.
- **Consultas relacionales optimizadas**: obtención de calendarios hijos directos mediante `parent_calendar_id` y construcción de jerarquías completas usando el campo path, demostrando prácticas de diseño NoSQL para estructuras jerárquicas.
- **Validación de integridad**: verificación de existencia del calendario padre antes de crear hijos, garantizando consistencia de datos en estructuras jerárquicas.

4.3.2. Endpoints

El servicio expone 11 endpoints bajo el prefijo `/v1/calendars`. A continuación, se detallan los parámetros de consulta aceptados por cada endpoint.

4.3.2.1. Posibles parámetros

POST `/v1/calendars`

Crea un nuevo calendario.

Parámetro	Tipo	Requerido	Descripción
body JSON	-	-	JSON con, al menos, los parámetros obligatorios de un calendario.

Ejemplo de uso: POST `/v1/calendars`

GET /v1/calendars/{calendar_id}

Obtiene un calendario específico por su ID.

Parámetro	Tipo	Requerido	Descripción
calendar_id	string	Sí	ID del calendario (ObjectId de MongoDB)

Ejemplo de uso: GET /v1/calendars/507f1f77bcf86cd799439013

PUT /v1/calendars/{calendar_id}

Actualiza un calendario existente. Solo el creador puede actualizar el calendario.

Parámetro	Tipo	Requerido	Descripción
calendar_id	string	Sí	ID del calendario a actualizar
body JSON	-	-	JSON con los parámetros de un calendario que se quieran actualizar.

Ejemplo de uso: PUT /v1/calendars/ 507f1f77bcf86cd799439013

DELETE /v1/ calendars/{calendar_id}

Elimina un calendario del sistema. Solo el creador puede eliminar el calendario.

Parámetro	Tipo	Requerido	Descripción
calendar_id	string	Sí	ID del calendario a eliminar.

Ejemplo de uso: DELETE /v1/calendars/507f1f77bcf86cd799439013

GET /v1/calendars/search/by-creator

Busca calendarios por creador.

Parámetro	Tipo	Requerido	Descripción
creator_external_id	string	Sí	External ID del usuario creador (OAuth). Filtra calendarios donde creator_external_id coincide con el valor proporcionado.

Ejemplo de uso:

GET /v1/calendars/search/by-creator?creator_external_id=google_123456789

GET /v1/calendars/search/by-keywords

Busca calendarios por palabras clave.

Parámetro	Tipo	Requerido	Descripción
keyword	string	Sí	Palabra clave a buscar. Busca en el array keywords de los calendarios (búsqueda case-sensitive)

Ejemplo de uso: GET /v1/calendars/search/by-keywords?k

GET /v1/calendars/search/by-visibility

Busca calendarios por visibilidad.

Parámetro	Tipo	Requerido	Descripción
visibility	string	Sí	Visibilidad del calendario. Valores permitidos: public, private, unlisted. Búsqueda case-sensitive

Ejemplo de uso: GET /v1/calendars/search/by-visibility?visibility=public

GET /v1/calendars/{calendar_id}/children

Obtiene los calendarios hijos directos de un calendario.

Parámetro	Tipo	Requerido	Descripción
calendar_id	string	Sí	ID del calendario padre. Retorna los calendarios donde parent_calendar_id coincide con el valor proporcionado.

Ejemplo de uso: GET /v1/calendars/507f1f77bcf86cd799439013/children

GET /v1/calendars/{calendar_id}/hierarchy

Parámetro	Tipo	Requerido	Descripción
calendar_id	string	Sí	ID del calendario raíz. Retorna el calendario y todos sus descendientes en estructura jerárquica

Ejemplo de uso: GET /v1/calendars/507f1f77bcf86cd799439013/hierarchy

GET /v1/calendars/search/by-text

Búsqueda full-text en calendarios (title, description, keywords).

Parámetro	Tipo	Requerido	Descripción
query	string	Sí	Término de búsqueda (case-insensitive)

Ejemplo de uso: GET /v1/calendars/search/by-text?query=universidad

GET /v1/calendars/search/by-creator-name

Busca calendarios por el campo denormalizado creator_display_name.

Parámetro	Tipo	Requerido	Descripción
creator_name	string	Sí	Nombre o parte del nombre del creador

Ejemplo de uso: GET /v1/calendars/search/by-creator-name?creator_name=JOGE

4.3.2.2. Formato de datos en petición y respuesta

4.3.2.2.1. Peticiones

Todos los endpoints POST y PUT utilizan el método correspondiente y pueden recibir datos mediante body JSON. Los endpoints GET y DELETE no requieren cuerpo (body) en ninguna petición.

4.3.2.2.2. Respuestas

Las respuestas del Calendar Service siguen un formato JSON consistente. A continuación, se detallan los schemas de respuesta para cada tipo de endpoint:

CalendarResponse (Response Body)

```
{
  "id": "string",
  "title": "string",
  "creator_external_id": "string",
  "creator_display_name": "string",
  "keywords": ["string"],
  "color": "string",
  "icon": "string | null",
  "parent_calendar_id": "string | null",
  "path": ["string"],
  "description": "string | null",
  "visibility": "string",
  "created_at": "datetime (ISO 8601)",
  "updated_at": "datetime (ISO 8601)",
  "subscriber_count": "integer"
}
```

CalendarHierarchy (Response Body - solo para endpoint /hierarchy)

```
{
  "calendar": {
    "id": "string",
    "title": "string",
    "creator_external_id": "string",
    "creator_display_name": "string",
    "keywords": ["string"],
    "color": "string",
    "icon": "string | null",
    "parent_calendar_id": "string | null",
    "path": ["string"],
    "description": "string | null",
    "visibility": "string",
    "created_at": "datetime (ISO 8601)",
    "updated_at": "datetime (ISO 8601)",
    "subscriber_count": "integer"
  },
  "children": [
    {
      "calendar": { ... },
      "children": [ ... ]
    }
  ]
}
```

ResponseMessage (Response Body - solo para DELETE)

```
{
  "message": "string"
}
```

Posibles códigos de respuesta

Código	Descripción
200 OK	Operación ejecutada correctamente. Retorna el recurso solicitado o confirmación de la acción.
201 Created	Calendario creado exitosamente. Retorna el calendario completo con su ID generado y campos auto-poblados (id, created_at, updated_at, subscriber_count, path).
400 Bad Request	Petición inválida. Los datos enviados no cumplen con el schema requerido, faltan parámetros obligatorios, el formato de ObjectId es inválido, o el calendario padre no existe.
404 Not Found	Recurso no encontrado. El calendario con el ID especificado no existe en la base de datos.
422 Unprocessable Entity	Error de validación de Pydantic. Los datos del body JSON no pasan las validaciones del schema (tipo de dato incorrecto, campo faltante, valor de visibility inválido, etc.).
500 Internal Server Error	Error en el servidor o en la conexión con MongoDB.

4.4. Event Service (puerto 8003)

4.4.1. Descripción

Microservicio especializado en gestionar el ciclo de vida de los eventos de la plataforma Basmati. Mantiene estado propio en MongoDB (colección events) y actúa como fuente de verdad para eventos, comentarios y adjuntos. Su responsabilidad incluye crear, leer, actualizar y eliminar eventos, administrar contenido asociado (comentarios y adjuntos) y emitir notificaciones cuando se añaden comentarios.

Su arquitectura está diseñada bajo el patrón **Repository–Service–API** para separar acceso a datos, lógica de negocio y exposición REST. Aprovecha campos **denormalizados** presentes en el esquema, como `calendar_title`, `creator_external_id` y `author_display_name`, permitiendo vistas y consultas relacionales sin realizar joins entre colecciones, lo que mejora la latencia y simplifica el diseño NoSQL. El acceso a MongoDB se realiza con el driver asíncrono **Motor**; se recomiendan índices en `calendar_id`, `start_time/end_time` y `comments.author_external_id`. La lógica valida reglas de negocio (fechas en UTC, `end_time` $\geq$ `start_time`) y normaliza identificadores. Tras añadir un comentario, el servicio comunica un evento de notificación a NotificationService mediante HTTP asíncrono con timeouts y manejo de errores no bloqueante.

El servicio implementa búsquedas parametrizadas eficientes por calendario y por rango de fechas. Las búsquedas por texto en títulos/descripciones pueden ser case-insensitive usando expresiones regulares de MongoDB y los resultados se limitan (p. ej., máximo 100) para prevenir sobrecargas de memoria y mantener respuestas ágiles. Entre sus responsabilidades principales se encuentran:

- **CRUD de eventos:** creación, lectura, actualización y eliminación con validaciones de negocio y normalización de fechas (UTC).
- **Gestión de comentarios:** inserción ordenada por `created_at`; al crear un comentario se emite una notificación tipo `NEW_COMMENT` hacia NotificationService sin afectar la persistencia si el envío falla.
- **Gestión de adjuntos:** almacenamiento de metadatos (`filename`, `url`, `size`, `mime_type`, `is_image`, `thumbnail_url`) sin manejar binarios en el servicio.
- **Búsquedas parametrizadas:** por `calendar_id` y por rango temporal (`start/end`), con ordenación por `start_time` y límites de resultados.
- **Consultas relacionales optimizadas:** obtención de usuarios que comentaron un evento y listado de eventos comentados por un usuario, explotando campos denormalizados en `comments`.
- **Observabilidad y robustez:** timeouts/reintentos acotados en llamadas externas, manejo claro de 400/404/422 y documentación automática vía Pydantic v2 con ejemplos realistas en OpenAPI.

4.4.2. Endpoints

El servicio expone **10 endpoints** bajo el prefijo `/v1/events`. A continuación, se detallan los parámetros de consulta aceptados por cada endpoint.

4.4.2.1. Posibles parámetros

POST `/v1/events`

Crea un nuevo evento.

Parámetro	Tipo	Requerido	Descripción
body JSON	-	-	JSON con, al menos, los parámetros obligatorios de un calendario.

Ejemplo de uso: POST `/v1/events`

GET /v1/events/{event_id}

Obtiene un evento específico por su ID.

Parámetro	Tipo	Requerido	Descripción
event_id	string	Sí	ID del evento solicitado.

Ejemplo de uso: GET /v1/events/123456789

PUT /v1/events/{event_id}

Modifica un evento específico por su ID.

Parámetro	Tipo	Requerido	Descripción
event_id	string	Sí	ID del evento modificado.
body JSON	-	-	JSON con los parámetros del evento a actualizar.

Ejemplo de uso: PUT /v1/events/123456789

DELETE /v1/events/{event_id}

Elimina un evento específico por su ID.

Parámetro	Tipo	Requerido	Descripción
event_id	string	Sí	ID del evento a eliminar.

Ejemplo de uso: DELETE /v1/events/123456789

POST /v1/events/{event_id}/comments

Crea un comentario en un evento por su ID.

Parámetro	Tipo	Requerido	Descripción
event_id	string	Sí	ID del evento en el que se añade el comentario.
body JSON	-	-	ID del comentarista y cuerpo del comentario.

Ejemplo de uso: POST /v1/events/123456789/comments

POST /v1/events/{event_id}/attachments

Adjunto un archivo al evento por su ID.

Parámetro	Tipo	Requerido	Descripción
event_id	string	Sí	ID del evento en el que se adjunta el archivo.
body JSON	-	-	Información sobre el archivo a adjuntar

Ejemplo de uso: POST /v1/events/123456789/attachments

GET /v1/events/{event_id}/comments/users

Recupera autores que comentaron en el evento por su ID.

Parámetro	Tipo	Requerido	Descripción
event_id	string	Sí	ID del evento solicitado.

Ejemplo de uso: GET /v1/events/123456789/comments/users

GET /v1/events/users/{user_external_id}/commented

Recupera los comentarios publicados por un usuario por su ID externa.

Parámetro	Tipo	Requerido	Descripción
user_external_id	string	Sí	ID externa (OAuth) del usuario del que se buscan los comentarios

Ejemplo de uso: GET /v1/events/users/google_123456789/commented

GET /v1/events/search/by-calendar

Lista eventos pertenecientes a un calendario (parametrized query 1).

Parámetro	Tipo	Requerido	Descripción
calendar_id	string	Sí	ID del calendario (MongoDB ObjectId como string)

Ejemplo de uso:

GET /v1/events/search/by-calendar?calendar_id=507f1f77bcf86cd799439011

GET /v1/events/search/by-date-range

Busca eventos dentro de un rango de fechas (parametrized query 2).

Parámetro	Tipo	Requerido	Descripción
start	datetime	Sí	Fecha inicio (ISO 8601)
end	datetime	Sí	Fecha fin (ISO 8601) — debe ser posterior a start

Ejemplo de uso:

GET /v1/events/search/by-date-range?start=2025-03-01T00:00:00Z&end=2025-03-31T23:59:59Z

GET /v1/events/search/by-text

Búsqueda full-text en eventos (title, description, location.address, location.place_name).

Parámetro	Tipo	Requerido	Descripción
query	string	Sí	Término de búsqueda (case-insensitive)

Ejemplo de uso: GET /v1/events/search/by-text?query=examen

GET /v1/events/search/by-calendar-title

Busca eventos por el campo denormalizado `calendar_title`.

Parámetro	Tipo	Requerido	Descripción
<code>calendar_title</code>	<code>string</code>	Sí	Título o parte del título del calendario.

Ejemplo de uso: GET /v1/events/search/by-calendar-title?calendar_title=Marketing

GET /v1/events/search/by-location

Búsqueda de eventos por ubicación geográfica.

Parámetro	Tipo	Requerido	Descripción
<code>location_query</code>	<code>string</code>	Sí	Término de búsqueda para ubicación.

Ejemplo de uso: GET /v1/events/search/by-location?location_query=Aula+Magna

4.4.2.2. Formato de datos en petición y respuesta

4.4.2.2.1. Peticiones

Todos los endpoints POST y PUT utilizan el método correspondiente y pueden recibir datos mediante body JSON. Los endpoints GET y DELETE no requieren cuerpo (body) en ninguna petición.

4.4.2.2.2. Respuestas

Las respuestas del Event Service siguen un formato JSON consistente. A continuación, se detallan los schemas de respuesta para cada tipo de endpoint:

EventResponse (Response Body)

```
{
  "id": "string",
  "calendar_id": "string",
  "calendar_title": "string",
  "creator_external_id": "string",
  "title": "string",
  "description": "string | null",
  "start_time": "datetime (ISO 8601)",
  "end_time": "datetime (ISO 8601)",
  "location": {
    "address": "string",
    "latitude": "float",
    "longitude": "float",
    "place_name": "string | null",
    "map_provider": "string"
  } | null,
  "attachments": [
    {
      "id": "string",
      "filename": "string",
```

```

    "url": "string",
    "size": "integer",
    "mime_type": "string",
    "uploaded_at": "datetime (ISO 8601)",
    "uploaded_by": "string",
    "is_image": "boolean",
    "thumbnail_url": "string | null"
  }
],
"comments": [
  {
    "id": "string",
    "author_external_id": "string",
    "author_display_name": "string",
    "text": "string",
    "created_at": "datetime (ISO 8601)"
  }
],
"visibility": "string",
"recurrence": {
  "pattern": "string",
  "interval": "integer",
  "days_of_week": ["integer"] | null,
  "end_date": "datetime (ISO 8601) | null",
  "exceptions": ["datetime (ISO 8601)"]
} | null,
"created_at": "datetime (ISO 8601)",
"updated_at": "datetime (ISO 8601)"
}

```

CommentCreate (Request Body)

```

{
  "author_external_id": "string",
  "author_display_name": "string",
  "text": "string"
}

```

AttachmentCreate (Request Body)

```

{
  "filename": "string",
  "url": "string",
  "size": "integer",
  "mime_type": "string",
  "uploaded_by": "string",
  "is_image": "boolean",
  "thumbnail_url": "string | null"
}

```

EventCommentAuthor (Response Body - solo para endpoint /comments/users)

```
{
  "author_external_id": "string",
  "author_display_name": "string",
  "comment_count": "integer"
}
```

ResponseMessage (ResponseBody – Solo DELETE)

```
{
  "message": "string"
}
```

Posibles códigos de respuesta

Código	Descripción
200 OK	Búsqueda ejecutada correctamente. Retorna lista de resultados (puede estar vacía si no hay coincidencias).
201 CREATED	Recurso creado exitosamente (POST /events, POST /events/{id}/comments, POST /events/{id}/attachments).
204 No Content	Eliminación completada (DELETE).
400 Bad Request	Validación de negocio fallida.
404 Not Found	Evento no encontrado.
422 Unprocessable Entity	Parámetros de consulta inválidos o faltantes (validación de Pydantic).
500 Internal Server Error	Error en el servidor o en la conexión con MongoDB.

4.5. Notification Service (puerto 8004)

4.5.1. Descripción

Microservicio especializado en **gestionar el sistema de notificaciones** de la plataforma Basmati. Mantiene estado propio en MongoDB (colección `notifications`) y actúa como receptor de eventos de otros servicios para generar notificaciones en tiempo real. Su responsabilidad incluye crear, leer, marcar como leídas y eliminar notificaciones destinadas a los usuarios de la plataforma.

Su arquitectura está diseñada bajo el patrón **Repository-Service-API** para separar acceso a datos, lógica de negocio y exposición REST. Aprovecha campos **desnormalizados** presentes en el esquema, como `recipient_external_id` (referencia al usuario OAuth), `related_event_id` y `related_calendar_id`, para mantener trazabilidad sin realizar joins entre colecciones, lo que mejora la latencia y simplifica el diseño NoSQL. El acceso a MongoDB se realiza con el driver asíncrono **Motor**, se recomiendan índices en `recipient_external_id`, `is_read` y `expires_at` (índice TTL para expiración automática). La lógica valida reglas de negocio como tipos de notificación permitidos (`NEW_COMMENT`, `EVENT_UPDATE`, `CALENDAR_INVITE`, `EVENT_REMINDER`) y normaliza identificadores. Tras crear o actualizar eventos, otros servicios (como **EventService**) comunican al **NotificationService** mediante HTTP asíncrono con `httpx` y manejo de errores no bloqueante.

El servicio implementa **búsquedas parametrizadas** eficientes por usuario y evento. Las búsquedas permiten filtrar por estado de lectura (`is_read`), tipo de notificación, evento relacionado y calendario relacionado. Los resultados se ordenan por fecha de creación descendente (más recientes primero) y se limitan para prevenir sobrecargas de memoria y mantener respuestas ágiles. Entre sus responsabilidades principales se encuentran:

- **CRUD de notificaciones:** creación, lectura, actualización (marcar como leída) y eliminación con validaciones de negocio y normalización de identificadores (`ObjectId` ↔ `string`).
- **Gestión de expiración automática:** uso de índice TTL en MongoDB sobre el campo `expires_at` para eliminar notificaciones vencidas sin intervención manual del servicio.
- **Búsquedas parametrizadas:** por `recipient_external_id` (usuario), `related_event_id` (evento), `related_calendar_id` (calendario), `type` (tipo de notificación) e `is_read` (estado de lectura), con ordenación por `created_at` descendente y límites de resultado.
- **Operaciones masivas:** marcado de todas las notificaciones de un usuario como leídas mediante `update_many` para optimizar operaciones batch.
- **Consultas relacionales optimizadas:** obtención de notificaciones filtrando por múltiples criterios simultáneamente (ej: notificaciones no leídas de un usuario sobre un evento específico), explotando campos **desnormalizados** para evitar joins.
- **Observabilidad y robustez:** manejo robusto de errores mediante try-catch con códigos HTTP estándar (200/201/400/404/500), validación automática con Pydantic v2 y logs descriptivos en llamadas externas.

4.5.2. Endpoints

El servicio expone **10 endpoints** bajo el prefijo `/v1/notifications`. A continuación, se detallan los parámetros de consulta aceptados por cada endpoint.

4.5.2.1. Posibles parámetros

POST /v1/notifications

Crea una nueva notificación.

Parámetro	Tipo	Requerido	Descripción
body JSON	-	-	JSON con los parámetros de la notificación

Ejemplo de uso: GET /v1/notifications

GET /v1/notifications/user/{user_id}

Obtiene todas las notificaciones de un usuario.

Parámetro	Tipo	Requerido	Descripción
user_id	string	Sí	External ID del usuario (OAuth). (ID del usuario receptor de las notificaciones)

Ejemplo de uso: GET /v1/notifications/user/google_123456789

GET /v1/notifications/{notification_id}

Obtiene una notificación específica por su ID.

Parámetro	Tipo	Requerido	Descripción
notification_id	string	Sí	ID de la notificación.

Ejemplo de uso: GET /v1/notifications/507f1f77bcf86cd799439013

PUT /v1/notifications/{notification_id}/read

Marca una notificación como leída.

Parámetro	Tipo	Requerido	Descripción
notification_id	string	Sí	ID de la notificación.

Ejemplo de uso: GET /v1/notifications/507f1f77bcf86cd799439013/read

PUT /v1/notifications/user/{user_id}/read-all

Marca todas las notificaciones de un usuario como leídas.

Parámetro	Tipo	Requerido	Descripción
user_id	string	Sí	External ID del usuario cuyas notificaciones se marcarán como leídas.

Ejemplo de uso: PUT /v1/notifications/user/google_123456789/read-all

GET /v1/notifications/search/unread

Busca notificaciones no leídas de un usuario (*Parametrized Query 1*).

Parámetro	Tipo	Requerido	Descripción
user_id	string	Sí	External ID del usuario. Filtra notificaciones donde is_read=false para el usuario especificado.

Ejemplo de uso: GET /v1/notifications/search/unread?user_id=google_123456789

GET /v1/notifications/search/by-event

Busca notificaciones relacionadas con un evento (*Parametrized Query 2*).

Parámetro	Tipo	Requerido	Descripción
event_id	string	Sí	External ID del usuario. Filtra notificaciones donde is_read=false para el usuario especificado.

Ejemplo de uso:

GET /v1/notifications/search/by-event?event_id=507f1f77bcf86cd799439011

GET /v1/notifications/search/by-type

Busca notificaciones por tipo (*Parametrized Query 3*).

Parámetro	Tipo	Requerido	Descripción
type	string	Sí	Tipo de notificación. Valores permitidos: NEW_COMMENT, EVENT_UPDATE, CALENDAR_INVITE, EVENT_REMINDER. Búsqueda case-sensitive .

Ejemplo de uso: GET /v1/notifications/search/by-type?type=EVENT_UPDATE

GET /v1/notifications/search/by-calendar

Busca notificaciones relacionadas con un calendario (*Parametrized Query 4*).

Parámetro	Tipo	Requerido	Descripción
calendar_id	string	Sí	ID del calendario (ObjectId). Busca notificaciones donde related_calendar_id coincide con el valor proporcionado.

Ejemplo de uso:

GET /v1/notifications/search/by-calendar?calendar_id=507f1f77bcf86cd799439012

DELETE /v1/notifications/{notification_id}

Elimina una notificación.

Parámetro	Tipo	Requerido	Descripción
notification_id	string	Sí	ID de la notificación a eliminar.

Ejemplo de uso: DELETE /v1/notifications/507f1f77bcf86cd799439013

4.5.2.2. Formato de datos en petición y respuesta

4.5.2.2.1. Peticiones

Todos los endpoints **POST** y **PUT** utilizan el método correspondiente y pueden recibir datos mediante **body JSON**. Los endpoints **GET** y **DELETE** no requieren cuerpo (body) en ninguna petición.

4.5.2.2.2. Respuestas

Las respuestas del **Notification Service** siguen un formato JSON consistente. A continuación, se detallan los **schemas** de respuesta para cada tipo de endpoint:

NotificationResponse (Response Body)

```
{
  "id": "string",
  "recipient_external_id": "string",
  "type": "string",
  "title": "string",
  "message": "string",
  "is_read": "boolean",
  "related_event_id": "string | null",
  "related_calendar_id": "string | null",
  "created_at": "datetime (ISO 8601)",
  "expires_at": "datetime (ISO 8601) | null"
}
```

ResponseMessage

```
{
  "message": "string"
}
```

Posibles códigos de respuesta

Código	Descripción
200 OK	Operación ejecutada correctamente. Retorna el recurso solicitado o confirmación de la acción.
201 Created	Notificación creada exitosamente. Retorna la notificación completa con su ID generado y campos auto-poblados (<code>id</code> , <code>created_at</code> , <code>is_read</code>).
400 Bad Request	Petición inválida. Los datos enviados no cumplen con el schema requerido, faltan parámetros obligatorios, o el formato de <code>ObjectId</code> es inválido.
404 Not Found	Recurso no encontrado. La notificación con el ID especificado no existe en la base de datos.
422 Unprocessable Entity	Error de validación de Pydantic. Los datos del body JSON no pasan las validaciones del schema (tipo de dato incorrecto, campo faltante, etc.).
500 Internal Server Error	Error en el servidor o en la conexión con MongoDB.

4.6. Integration Service (puerto 8006)

4.6.1. Descripción

Este servicio actúa como un puente de **importación unidireccional** para calendarios externos. Su único propósito es leer datos de **Google Calendar** y **TeamUp** y copiarlos para crearlos dentro de la plataforma. Este servicio actualmente **no está completamente disponible debido a la falta de autenticación** (OAuth), necesaria para la importación de calendarios de Google Calendar.

Este servicio tiene dos características fundamentales:

1. **Importación como "Snapshot" (foto fija):** el servicio no crea una sincronización continua. En su lugar, toma una "foto" del calendario externo en el momento exacto de la importación. Todos los eventos y configuraciones de ese instante son los que se copian.
2. **Re-importación sin restricciones:** un usuario puede importar el mismo calendario (ej. "Calendario de Festivos" de Google) varias veces. El sistema lo permite sin restricciones, lo que podría llevar a tener calendarios duplicados si el usuario no tiene cuidado. Esto también podría ser un mecanismo manual para "actualizar" un calendario (borrando el antiguo e importando el nuevo).

4.6.2. Endpoints

POST /google/import

Importa uno o más calendarios desde Google Calendar.

Parámetro	Tipo	Requerido	Descripción
user_external_id	string	Sí	External ID del usuario que realiza la importación.
google_access_token	string	Sí	Token de acceso OAuth 2.0 de Google. Se obtiene mediante el flujo de autenticación de Google. Debe tener permisos para leer calendarios del usuario.
calendar_ids	array[string]	-	Lista de IDs específicos de calendarios de Google a importar. Si es null o se omite, se importan todos los calendarios accesibles del usuario.

Ejemplo de uso: POST /v1/integrations/google/import

POST /teamup/import

Importa uno o más calendarios (sub-calendarios) desde Teamup.

Parámetro	Tipo	Requerido	Descripción
user_external_id	string	Sí	External ID del usuario que realiza la importación.
calendar_keys	array[string]	Sí	Lista de calendar keys de Teamup a importar. Cada key identifica un calendario/sub-calendario en Teamup.
teamup_api_key	string	-	API Key de Teamup. Si no se proporciona, se usa la API Key configurada en el servidor por defecto. Solo necesario si se quiere usar una API Key diferente.

Ejemplo de uso: POST /v1/integrations/teamup/import

4.6.2.1. Formato de datos en petición y respuesta

4.6.2.1.1. Peticiones

Todos los endpoints **POST** utilizan el método correspondiente y pueden recibir datos mediante **body JSON**.

4.6.2.1.2. Respuestas

Las respuestas del Integration Service siguen un formato JSON consistente.

ImportResponse (Response Body)

```
{
  "success": "boolean",
  "message": "string",
  "imported_sources": [
    {
      "external_id": "string",
      "basmati_calendar_id": "string"
    }
  ],
  "errors": ["string"]
}
```

5. Instalación – Docker

5.1. Scripts de Docker

El archivo más importante es `docker-compose.yml`, que es el que define la estructura basada en microservicios de la arquitectura. Su objetivo es **levantar, configurar y conectar múltiples contenedores** (seis, en este caso) para que funcionen como una sola aplicación cohesiva.

```
services:
  # API Gateway
  api-gateway:
    build:
      context: .
      dockerfile: api-gateway/Dockerfile
    container_name: basmati-api-gateway
    ports:
      - "8000:8000"
    environment:
      - SERVICE_PORT=8000
      - USER_SERVICE_URL=http://user-service:8001
      - CALENDAR_SERVICE_URL=http://calendar-service:8002
      - EVENT_SERVICE_URL=http://event-service:8003
      - NOTIFICATION_SERVICE_URL=http://notification-service:8004
      - INTEGRATION_SERVICE_URL=http://integration-service:8006
    depends_on:
      - user-service
      - calendar-service
      - event-service
      - notification-service
      - integration-service
    networks:
      - basmati-network

  # User Service
  user-service:
    build:
      context: .
      dockerfile: user_service/Dockerfile
    container_name: basmati-user-service
    ports:
      - "8001:8001"
    environment:
      - MONGO_URI=${MONGO_URI}
      - SERVICE_PORT=8001
      - DATABASE_NAME=basmati
    networks:
      - basmati-network

  # Calendar Service
  calendar-service:
```

```

build:
  context: .
  dockerfile: calendar_service/Dockerfile
container_name: basmati-calendar-service
ports:
  - "8002:8002"
environment:
  - MONGO_URI=${MONGO_URI}
  - SERVICE_PORT=8002
  - DATABASE_NAME=basmati
  - USER_SERVICE_URL=http://user-service:8001
depends_on:
  - user-service
networks:
  - basmati-network

# Event Service
event-service:
  build:
    context: .
    dockerfile: event_service/Dockerfile
  container_name: basmati-event-service
  ports:
    - "8003:8003"
  environment:
    - MONGO_URI=${MONGO_URI}
    - SERVICE_PORT=8003
    - DATABASE_NAME=basmati
    - NOTIFICATION_SERVICE_URL=http://notification-service:8004
    - CALENDAR_SERVICE_URL=http://calendar-service:8002
  depends_on:
    - calendar-service
    - notification-service
  networks:
    - basmati-network

# Notification Service
notification-service:
  build:
    context: .
    dockerfile: notification_service/Dockerfile
  container_name: basmati-notification-service
  ports:
    - "8004:8004"
  environment:
    - MONGO_URI=${MONGO_URI}
    - SERVICE_PORT=8004
    - DATABASE_NAME=basmati
    - USER_SERVICE_URL=http://user-service:8001
    - EMAIL_SERVICE_API_KEY=${EMAIL_SERVICE_API_KEY:-}
  depends_on:
    - user-service

```

```

networks:
  - basmati-network

# Integration Service
integration-service:
  build:
    context: .
    dockerfile: integration_service/Dockerfile
  container_name: basmati-integration-service
  ports:
    - "8006:8006"
  environment:
    - MONGO_URI=${MONGO_URI}
    - SERVICE_PORT=8006
    - DATABASE_NAME=basmati
    - CALENDAR_SERVICE_URL=http://calendar-service:8002
    - EVENT_SERVICE_URL=http://event-service:8003
    - GOOGLE_CALENDAR_API_KEY=${GOOGLE_CALENDAR_API_KEY:-}
    - TEAMUP_API_KEY=${TEAMUP_API_KEY:-}
  depends_on:
    - calendar-service
    - event-service
  networks:
    - basmati-network

networks:
  basmati-network:
    driver: bridge

```

Luego **dentro de cada microservicio existe un *Dockerfile***, el cual inicia la construcción a partir de una imagen base de Python, exactamente *python:3.11-slim*, la cual es una versión ligera, minimizada, provocando que esta imagen sea mucho más pequeña. Luego descarga todas las dependencias, las compartidas y las propias. Por último, expone en su puerto correspondiente su actividad, al cual se redirige la *api-gateway* cuando sea necesario.

5.2. Ejecución del contenedor

Para ejecutar la aplicación, el comando principal que usaremos es `docker-compose`. Este comando lee tu archivo `docker-compose.yml` y orquesta todos los servicios definidos en él.

Debes ejecutar todos estos comandos desde la carpeta `basmati/app/backend`, que es donde se encuentra tu archivo `docker-compose.yml`.

5.2.1. Ejecutar toda la aplicación (todos los servicios) - RECOMENDADO

El comando principal para levantar toda la arquitectura es `docker-compose up`.

Asegúrate de estar en el directorio correcto:

```
cd /ruta/a/basmati/app/backend
```

Este comando construye las imágenes (si no existen) y las inicia. Verás todos los logs de todos los contenedores en tu terminal. Es útil para debuggear el arranque.

```
docker-compose up
```

Esta es la **forma más común de hacerlo**. -d significa "detached". Los contenedores se ejecutan en segundo plano y la terminal queda libre.

```
docker-compose up -d
```

Cabe destacar que, antes de correr todos los contenedores, es recomendable (especialmente en un entorno de desarrollo) ejecutar el comando:

```
docker-compose build --no-cache
```

Esto asegura que las imágenes de Docker usan los últimos archivos de código disponibles en el proyecto.

6. Cambios en la arquitectura

6.1. Cambio de tecnología Cloud – AWS

El motivo principal para el cambio fue la dificultad para **obtener la cuenta gratuita** (Free Tier). El proceso para acceder y utilizar plenamente la capa gratuita de GCP resultó ser **innecesariamente complejo**. Las barreras de verificación, las restricciones geográficas en métodos de pago o la ambigüedad en los límites de uso hicieron que el acceso inicial fuera complicado de gestionar, dificultando la experimentación rápida y el onboarding de nuestro equipo.

Frente a una plataforma que complica tanto la facturación como el acceso inicial, la decisión de buscar una alternativa más abierta y madura se hizo inevitable. La migración a Amazon Web Services (AWS) emergió debido a que estaba ya entre nuestras opciones y que es la elección más simple y accesible.

6.1.1. Amazon EC2

Dentro del vasto ecosistema de AWS, **Amazon Elastic Compute Cloud (EC2)** se identificó inmediatamente como el pilar central para nuestra infraestructura de cómputo. EC2 es el servicio que proporciona capacidad informática segura y de tamaño modificable en la nube; en esencia, nos permite alquilar los servidores virtuales donde se ejecutarían nuestras aplicaciones.

Esta elección resolvió directamente las frustraciones que experimentamos con GCP:

1. **Claridad y acceso al free tier:** la principal ventaja fue la transparencia de la capa gratuita de AWS. A diferencia de las barreras encontradas anteriormente, AWS ofrece de forma explícita **750 horas mensuales de una instancia t2.micro** (o t3.micro, según la región) durante los primeros 12 meses.
2. **Madurez y ecosistema (el más "utilizado"):** EC2 es, posiblemente, el servicio IaaS (Infraestructura como Servicio) más veterano y probado del mercado. Esta madurez no solo se traduce en fiabilidad, sino en un ecosistema de soporte inmenso. Cualquier reto de configuración, integración o despliegue que enfrentamos ya había sido resuelto y documentado extensamente por la comunidad global, facilitando enormemente el *onboarding* del equipo.
3. **Simplicidad fundamental (el más "básico" y "accesible"):** aunque EC2 ofrece una profunda complejidad para configuraciones avanzadas, su concepto fundamental es simple: seleccionar una imagen de máquina (AMI), elegir un tamaño de instancia y lanzarla.

En resumen, EC2 representó la puerta de entrada sin fricciones que buscábamos: un servicio robusto y accesible, con un coste inicial nulo y predecible, que nos permitió centrarnos en el desarrollo de nuestro proyecto y no en la burocracia de la plataforma.